SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43
NAME: E.HEMANTH REDDY
REG.NO.: 192372098

COURSE OUTCOME COVERED

CO 3: Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

Assessment Tool : Scenario-Based Assessment

Weightage: 40%

QUESTIONS: **Total Marks: 30**

Question 1

Suppose that a college wants to develop an online student registration system where students submit their details through a web form, and the data is processed using a Java Servlet.
Explain the architecture of a servlet-based application. Design and implement a servlet to handle form data using both GET and POST methods. Describe the servlet life cycle (init, service, destroy) and justify which method is more appropriate for handling sensitive data.

Question 2

Suppose that a website requires users to log in and maintain their session across multiple pages after authentication.
Design a session management system using HttpSession and Cookies in Servlets. Explain how session tracking works in web applications and compare session-based tracking with cookies.

Question 3

Suppose that an e-commerce platform needs to store and retrieve product details from a database using a Java-based web application.
Design a JDBC-based application to connect to a database, insert product data, and retrieve it using SQL queries. Explain the JDBC architecture and describe the steps involved in database connectivity along with proper exception handling.

Code of Conduct:

*I E.HEMANTH REDDY**(192372098)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: E.HEMANTH REDDY

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues	Identifies most key issues with minor	Identifies limited issues; some irrelevant	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some	Makes weak decisions with limited justification	No clear decisions made or rationale
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly	Disorganized and difficult to understand

ANSWERS

Question-1

Student Details

Name

G. Chakradhar

Register Number

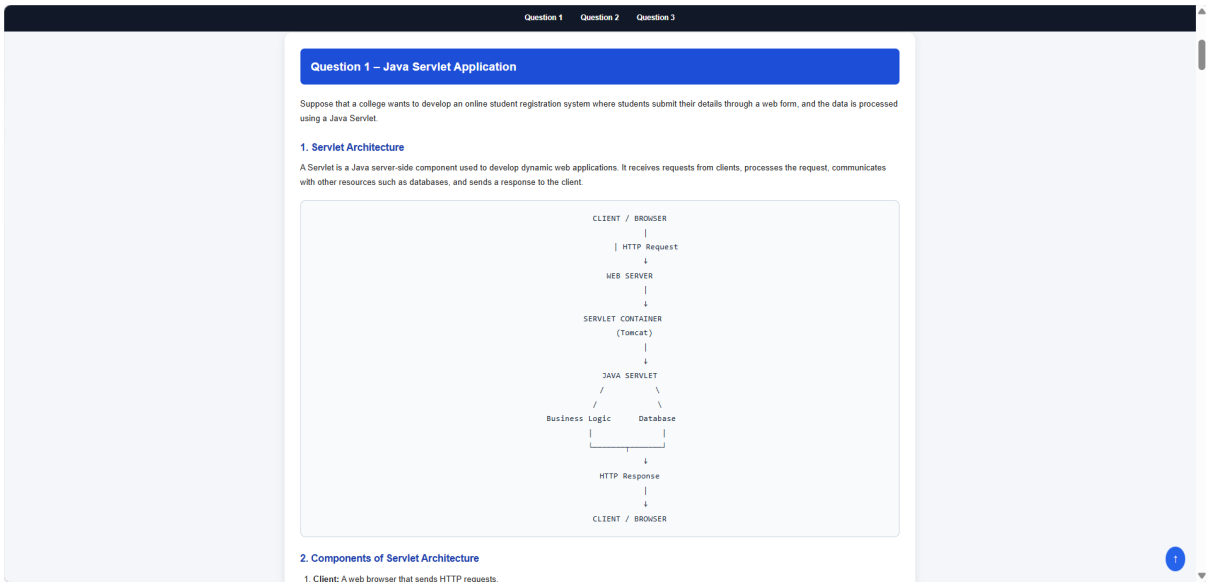
192311439

Course

Internet Programming

Course Code

CSA43



2. Components of Servlet Architecture

1. **Client:** A web browser that sends HTTP requests.
2. **Web Server:** Receives HTTP requests from the client.
3. **Servlet Container:** Manages servlet objects and their lifecycle. Apache Tomcat is a common example.
4. **Servlet:** Processes requests and generates responses.
5. **Database:** Stores student registration information.

3. Student Registration HTML Form

```
<!DOCTYPE html>
<html>

<head>
    <title>Student Registration</title>
</head>

<body>

<h2>Student Registration</h2>

<form action="StudentServlet" method="post">

    Name:
    <input type="text" name="name">
    <br><br>

    Register Number:
    <input type="text" name="regno">
    <br><br>

    Age:
    <input type="number" name="age">
    <br><br>

    Email:
    <input type="email" name="email">
    <br><br>

    <input type="submit" value="Register">

</form>

</body>
</html>
```



```
</html>
```

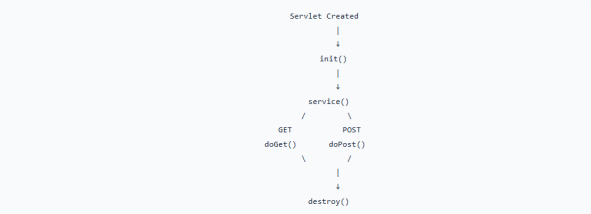
4. Servlet Using GET and POST

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class StudentServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response)
                         throws ServletException, IOException {

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        String name = request.getParameter("name");
        String regno = request.getParameter("regno");
        String age = request.getParameter("age");
        String email = request.getParameter("email");

        out.println("<h2>Student Registration - GET</h2>");

        out.println("<p>Name: " + name + "</p>");
        out.println("<p>Register Number: " + regno + "</p>");
        out.println("<p>Age: " + age + "</p>");
        out.println("<p>Email: " + email + "</p>");
    }


    @Override
    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
                          throws ServletException, IOException {

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        String name = request.getParameter("name");
        String regno = request.getParameter("regno");
        String age = request.getParameter("age");
        String email = request.getParameter("email");
```



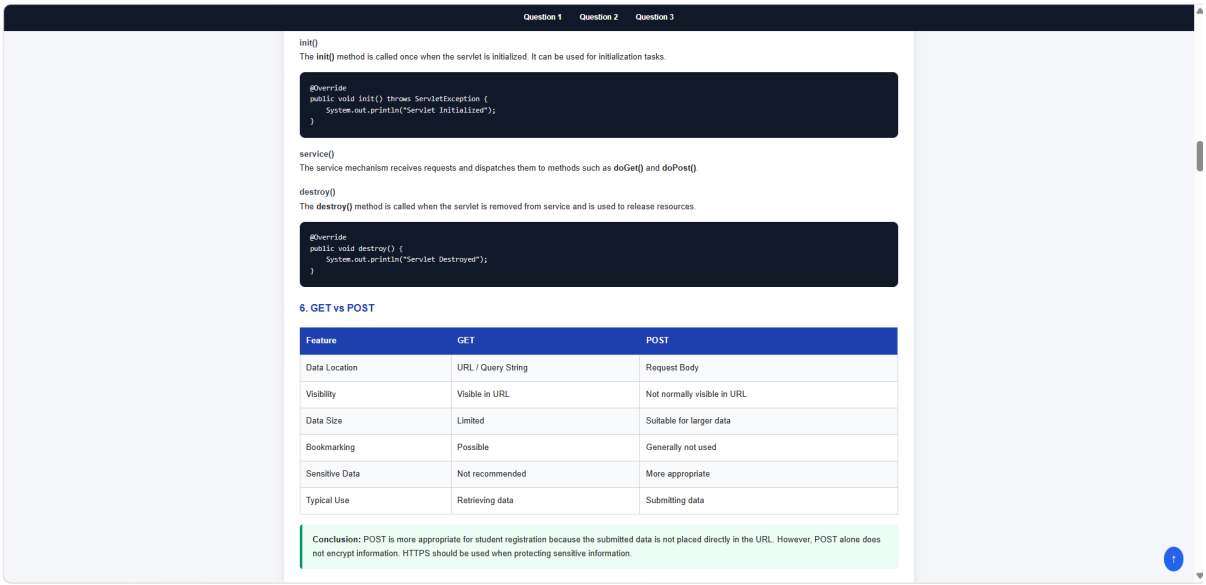
```
                          throws ServletException, IOException {

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        String name = request.getParameter("name");
        String regno = request.getParameter("regno");
        String age = request.getParameter("age");
        String email = request.getParameter("email");

        out.println("<h2>Registration Successful</h2>");

        out.println("<p>Name: " + name + "</p>");
        out.println("<p>Register Number: " + regno + "</p>");
        out.println("<p>Age: " + age + "</p>");
        out.println("<p>Email: " + email + "</p>");
    }
}
```

5. Servlet Life Cycle



init()
The **init()** method is called once when the servlet is initialized. It can be used for initialization tasks.

```
@Override
```



Question 1 Question 2 Question 3

init()
The **init()** method is called once when the servlet is initialized. It can be used for initialization tasks.

```
@Override
public void init() throws ServletException {
    System.out.println("Servlet Initialized");
}
```

service()
The service mechanism receives requests and dispatches them to methods such as **doGet()** and **doPost()**.

destroy()
The **destroy()** method is called when the servlet is removed from service and is used to release resources.

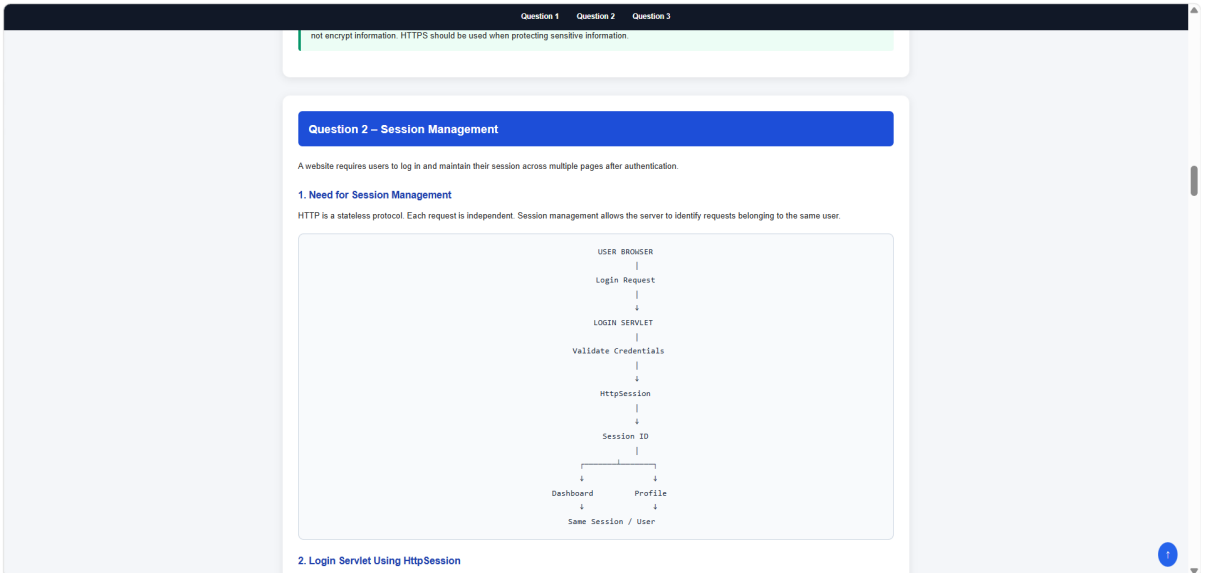
```
@Override
public void destroy() {
    System.out.println("Servlet Destroyed");
}
```

6. GET vs POST

Feature	GET	POST
Data Location	URL / Query String	Request Body
Visibility	Visible in URL	Not normally visible in URL
Data Size	Limited	Suitable for larger data
Bookmarking	Possible	Generally not used
Sensitive Data	Not recommended	More appropriate
Typical Use	Retrieving data	Submitting data

Conclusion: POST is more appropriate for student registration because the submitted data is not placed directly in the URL. However, POST alone does not encrypt information. HTTPS should be used when protecting sensitive information.

Question-2



Question 1 Question 2 Question 3

not encrypt information. HTTPS should be used when protecting sensitive information.

Question 2 – Session Management

A website requires users to log in and maintain their session across multiple pages after authentication.

1. Need for Session Management

HTTP is a stateless protocol. Each request is independent. Session management allows the server to identify requests belonging to the same user.

```
                    USER BROWSER
                         |
                   Login Request
                         |
                         ↓
                   LOGIN SERVLET
                         |
                 Validate Credentials
                         |
                         ↓
                    HttpSession
                         |
                         ↓
                     Session ID
                         |
                    ┌────┴─────┐
                    ↓          ↓
               Dashboard    Profile
                    ↓          ↓
                Same Session / User
```

2. Login Servlet Using HttpSession

2. Login Servlet Using HttpSession

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class LoginServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
                          throws ServletException, IOException {

        String username =
            request.getParameter("username");

        String password =
            request.getParameter("password");

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        if (username.equals("admin") &&
            password.equals("1234")) {

            HttpSession session =
                request.getSession();

            session.setAttribute(
                "username", username);

            session.setMaxInactiveInterval(
                30 * 60);

            response.sendRedirect("home");

        } else {

            out.println(
                "<h3>Invalid username or password</h3>");
        }
    }
}
```

3. Accessing Session Data

Question 1 Question 2 Question 3

3. Accessing Session Data

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class HomeServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response)
                         throws ServletException, IOException {

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        HttpSession session =
            request.getSession(false);

        if (session != null &&
            session.getAttribute("username") != null) {

            String username =
                (String) session.getAttribute("username");

            out.println(
                "<h2>Welcome " + username + "</h2>");

            out.println(
                "<p>You are logged in.</p>");

        } else {

            out.println(
                "<h2>Please login first.</h2>");
        }
    }
}
```

4. Logout Servlet

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class LogoutServlet extends HttpServlet {
```

Question 1 Question 2 Question 3

```
}
```

4. Logout Servlet

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class LogoutServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response)
                         throws ServletException, IOException {

        HttpSession session =
            request.getSession(false);

        if (session != null) {
            session.invalidate();
        }

        response.sendRedirect("login.html");
    }
}
```

5. Cookies

A Cookie is a small piece of information stored by the browser and sent back to the server with subsequent requests.

Creating a Cookie

```
Cookie userCookie =
    new Cookie("username", username);

userCookie.setMaxAge(60 * 60);

response.addCookie(userCookie);
```

Reading a Cookie

```
Cookie[] cookies = request.getCookies();

if (cookies != null) {
```

```
        response.sendRedirect("login.html");
    }
}
```

5. Cookies

A Cookie is a small piece of information stored by the browser and sent back to the server with subsequent requests.

Creating a Cookie

```
Cookie userCookie =
    new Cookie("username", username);

userCookie.setMaxAge(60 * 60);

response.addCookie(userCookie);
```

Reading a Cookie

```
Cookie[] cookies = request.getCookies();

if (cookies != null) {

    for (Cookie cookie : cookies) {

        if (cookie.getName().equals("username")) {

            String username = cookie.getValue();

            System.out.println(
                "User: " + username);
        }
    }
}
```

6. Session Tracking

```
1. User Login
        ↓
2. Server Creates HttpSession
        ↓
3. Session ID is Assigned
```

Question 1 Question 2 Question 3

```
}
```

6. Session Tracking

```
1. User Login
        ↓
2. Server Creates HttpSession
        ↓
3. Session ID is Assigned
        ↓
4. Browser Keeps Session Identifier
        ↓
5. Browser Sends Identifier
   With Future Requests
        ↓
6. Server Identifies User Session
        ↓
7. User Remains Logged In
```

7. HttpSession vs Cookies

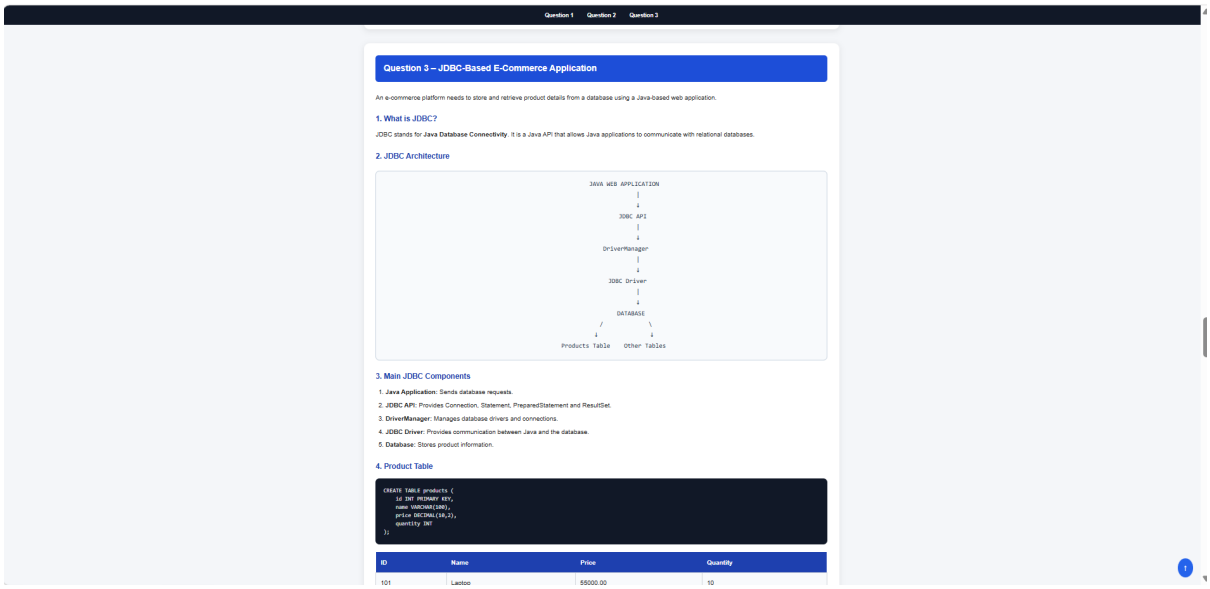
Feature	HttpSession	Cookies
Storage	Mainly Server-Side	Client/Browser-Side
Main Purpose	Maintain Session State	Store Small Client Data
Security	Better for Server-Side State	Should not directly store sensitive data
Lifetime	Controlled by Session Timeout	Controlled by Cookie Expiry
Example	Logged-in User	Language Preference

Security Improvements:

- Use HTTPS.
- Use secure session management.
- Use HttpOnly and Secure cookie attributes where appropriate.
- Invalidate sessions during logout.
- Set a suitable session timeout.
- Do not store passwords in cookies.

Conclusion: HttpSession is suitable for maintaining server-side authentication state across multiple pages. Cookies can support session identification and store small client-side information.

Question-3



Question 1 Question 2 Question 3

Question 3 – JDBC-Based E-Commerce Application

An e-commerce platform needs to store and retrieve product details from a database using a Java-based web application.

1. What is JDBC?

JDBC stands for **Java Database Connectivity**. It is a Java API that allows Java applications to communicate with relational databases.

2. JDBC Architecture

```
JAVA WEB APPLICATION
        |
        ↓
    JDBC API
        |
        ↓
  DriverManager
        |
        ↓
   JDBC Driver
        |
        ↓
    DATABASE
      /   \
     ↓     ↓
Products Table   Other Tables
```

3. Main JDBC Components

1. **Java Application:** Sends database requests.
2. **JDBC API:** Provides Connection, Statement, PreparedStatement and ResultSet.
3. **DriverManager:** Manages database drivers and connections.
4. **JDBC Driver:** Provides communication between Java and the database.
5. **Database:** Stores product information.

4. Product Table

```
CREATE TABLE products (
    id INT PRIMARY KEY,
    name VARCHAR(100),
    price DECIMAL(10,2),
    quantity INT
);
```

ID	Name	Price	Quantity
101	Laptop	55000.00	10

Question 1 Question 2 Question 3

1. **Java Application**: Sends database requests.
2. **JDBC API**: Provides Connection, Statement, PreparedStatement and ResultSet.
3. **DriverManager**: Manages database drivers and connections.
4. **JDBC Driver**: Provides communication between Java and the database.
5. **Database**: Stores product information.

4. Product Table

```
CREATE TABLE products (
    id INT PRIMARY KEY,
    name VARCHAR(100),
    price DECIMAL(10,2),
    quantity INT
);
```

ID	Name	Price	Quantity
101	Laptop	55000.00	10
102	Keyboard	1500.00	25
103	Mouse	800.00	30

5. Steps in JDBC Connectivity

1. Import JDBC packages.
2. Load/register the JDBC driver.
3. Establish database connection.
4. Create PreparedStatement.
5. Execute SQL query.
6. Process ResultSet.
7. Close resources.
8. Handle exceptions.

6. Insert Product Data

```
import java.io.*;
import java.sql.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class ProductServlet extends HttpServlet {

    private static final String URL =
        "jdbc:mysql://localhost:3306/ecommerce";

    private static final String USER = "root";

    private static final String PASSWORD = "root";

    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
```

Question 1 Question 2 Question 3

```
        catch (ClassNotFoundException e) {

            out.println(
                "<h3>JDBC Driver not found.</h3>");
        }

        catch (SQLException e) {

            out.println(
                "<h3>Database Error: " +
                e.getMessage() +
                "</h3>");
        }
    }
}
```

7. Retrieve Product Details

```
import java.io.*;
import java.sql.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class ViewProductsServlet extends HttpServlet {

    private static final String URL =
        "jdbc:mysql://localhost:3306/ecommerce";

    private static final String USER = "root";

    private static final String PASSWORD = "root";


    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response)
                         throws ServletException, IOException {

        response.setContentType("text/html");

        PrintWriter out =
            response.getWriter();


        String sql =
            "SELECT id, name, price, quantity " +
            "FROM products";


        try {

            Class.forName(
                "com.mysql.cj.jdbc.Driver");


            try {
                Connection con =
                    DriverManager.getConnection(
                        URL, USER, PASSWORD);

                PreparedStatement ps =
                    con.prepareStatement(sql);

                ResultSet rs =
                    ps.executeQuery()
```

Question 1 Question 2 Question 3

```
            out.println(
                "<h3>JDBC Driver not found.</h3>");
        }

        catch (SQLException e) {

            out.println(
                "<h3>Database Error: " +
                e.getMessage() +
                "</h3>");
        }
    }
}
```

8. Why PreparedStatement?

PreparedStatement allows values to be supplied separately from the SQL statement. This improves parameter handling and helps protect the application against SQL injection.

```
String sql =
    "INSERT INTO products " +
    "(id, name, price, quantity) " +
    "values (?, ?, ?, ?)";

PreparedStatement ps =
    con.prepareStatement(sql);

ps.setInt(1, id);
ps.setString(2, name);
ps.setDouble(3, price);
ps.setInt(4, quantity);
```

9. JDBC Workflow

10. Exception Handling

JDBC applications must handle database-related exceptions such as invalid credentials, connection failure, invalid SQL, missing drivers, and duplicate records.

```
try {
    // JDBC operations
}
catch (ClassNotFoundException e) {
    // Driver error
}
catch (SQLException e) {
    // Database error
}
```

Conclusion: JDBC provides a standard mechanism for connecting a Java web application with a database. PreparedStatement can be used for parameterized SQL operations, while ResultSet is used to process retrieved records.

Overall Conclusion

Question	Technology	Purpose
Question 1	Java Servlet	Process student registration form
Question 2	HttpSession + Cookies	Maintain logged-in user state
Question 3	JDBC	Store and retrieve product information

Key Points:

- Servlets process HTTP requests and generate responses.
- POST is preferred for form submission containing sensitive information, together with HTTPS for actual confidentiality.
- HttpSession maintains server-side session information across multiple requests.

Question 1 Question 2 Question 3

10. Exception Handling

JDBC applications must handle database-related exceptions such as invalid credentials, connection failure, invalid SQL, missing drivers, and duplicate records.

```
try {
    // JDBC operations
}
catch (ClassNotFoundException e) {
    // Driver error
}
catch (SQLException e) {
    // Database error
}
```

Conclusion: JDBC provides a standard mechanism for connecting a Java web application with a database. PreparedStatement can be used for parameterized SQL operations, while ResultSet is used to process retrieved records.

Overall Conclusion

Question	Technology	Purpose
Question 1	Java Servlet	Process student registration form
Question 2	HttpSession + Cookies	Maintain logged-in user state
Question 3	JDBC	Store and retrieve product information

Key Points:

- Servlets process HTTP requests and generate responses.
- POST is preferred for form submission containing sensitive information, together with HTTPS for actual confidentiality.
- HttpSession maintains server-side session information across multiple requests.
- Cookies store small pieces of information on the client and can support session identification.
- JDBC connects Java applications to relational databases.
- PreparedStatement provides parameterized SQL execution.
- Try-with-resources helps automatically close JDBC resources.